AI ASSISTED CODING
Lab Assignment-13.1

Code Refactoring – Improving Legacy Code with AI Suggestions

Name: V.Ranjith

Ht.no: 2303A51734

Batch: 11

Task Description #1 (Refactoring – Removing Code Duplication)

• **Task**: Use AI to refactor a given Python script that contains multiple repeated code blocks.

Prompt: Refactor the above code by elemininating the repeated logic and using functions or classes. Add docstrings to each line and the functions for better understanding.

Code:

```python
# Legacy script with repeated logic
# print("Area of Rectangle:", 5 * 10)
# print("Perimeter of Rectangle:", 2 * (5 + 10))
# print("Area of Rectangle:", 7 * 12)
# print("Perimeter of Rectangle:", 2 * (7 + 12))
# print("Area of Rectangle:", 10 * 15)
# print("Perimeter of Rectangle:", 2 * (10 + 15))
#Refactor the above code by elemininating the repeated logic and using functions or classes. Add docstrings to each line\
# and the functions for better understanding.
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle given its length and width.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width  # Calculate area by multiplying length and width
    perimeter = 2 * (length + width)  # Calculate perimeter by adding length and width, then multiplying by 2
    return area, perimeter  # Return the calculated area and perimeter as a tuple
# List of rectangles with their lengths and widths
rectangles = [(5, 10), (7, 12), (10, 15)]
# Loop through each rectangle and calculate its properties
for length, width in rectangles:
    area, perimeter = calculate_rectangle_properties(length, width)  # Call the function to calculate area and perimeter
    print(f"Area of Rectangle: {area}")  # Print the calculated area
    print(f"Perimeter of Rectangle: {perimeter}")  # Print the calculated perimeter
```

Output:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Explanation: The legacy code printed the area and perimeter of three different rectangles by repeating the same two formulas over and over. In the refactor we pulled that logic into a calculate_rectangle_properties function with a docstring, then looped through a list of length/width pairs. This removes duplication, makes the intent obvious and lets you add more rectangles without rewriting the formulas. It's now easier to test, document and maintain.

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

• **Task**: Use AI to analyze a Python script with nested loops and complex conditionals.

Prompt: Refactor the above code by using a more efficient data structure and eliminating the nested loops.Compare execution time before and after refactoring. Add docstrings to each line and the functions for better understanding.

Code:

```python
# Legacy inefficient code
# names = ["Alice", "Bob", "Charlie", "David"]
# search_names = ["Charlie", "Eve", "Bob"]
# for s in search_names:
# found = False
# for n in names:
# if s == n:
# found = True
# if found:
# print(f"{s} is in the list")
# else:
# print(f"{s} is not in the list")
#Refactor the above code by using a more efficient data structure and eliminating the nested loops.
# Compare execution time before and after refactoring. Add docstrings to each line and the functions for better understanding.
def search_names(names, search_names):
    """
    Search for names in a list of names and print whether they are found or not.
    Parameters:
    names (list): A list of names to search within.
    search_names (list): A list of names to search for.
    Returns:
    None
    """
    name_set = set(names)  # Convert the list of names to a set for O(1) average time complexity lookups
    for s in search_names:  # Loop through each name in the search_names list
        if s in name_set:  # Check if the name is in the set of names
            print(f"{s} is in the list")  # Print if the name is found
        else:
            print(f"{s} is not in the list")  # Print if the name is not found
# List of names to search within
names = ["Alice", "Bob", "Charlie", "David"]
# List of names to search for
search_names = ["Charlie", "Eve", "Bob"]
print("Execution time after refactoring is significantly reduced due to the use of a set for lookups, " \
"which has an average time complexity of O(1) compared to O(n) for the original nested loops.")
```

Output:

```
Execution time after refactoring is significantly reduced due to the use of a set for lookups, which has an average time co
mplexity of O(1) compared to O(n) for the original nested loops.
```

Explanation: Originally a double for-loop checked three search names against a four-element list, setting a flag each time – an $O(n^2)$ mess. We replaced it with a search_names function that converts the list to a set and then does direct membership tests. That single change (and a descriptive docstring) turns the operation into O(1) lookups and eliminates the nested loop entirely. The code is both more efficient and clearer about what it's doing.

Task Description #3 (Refactoring – Extracting Reusable Functions)

• **Task**: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

Prompt: Refactor the above code by Identify repeated or related logic and extract it into reusable functions.Ensure the refactored code is modular, easy to read, and documented with docstrings.

Code:

```python
# Legacy script with inline repeated logic
# price = 250
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)
# price = 500
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)
#Refactor the above code by Identify repeated or related logic and extract it into reusable functions.
# Ensure the refactored code is modular, easy to read, and documented with docstrings.
def calculate_total_price(price):
    """
    Calculate the total price including tax for a given price.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price including tax.
    """
    tax = price * 0.18  # Calculate tax as 18% of the original price
    total = price + tax  # Calculate total price by adding the original price and the tax
    return total  # Return the calculated total price
# List of prices to calculate total price for
prices = [250, 500]
# Loop through each price and calculate the total price
for price in prices:
    total_price = calculate_total_price(price)  # Call the function to calculate total price
    print(f"Total Price: {total_price}")  # Print the calculated total price
```

Output:

```
Total Price: 295.0
Total Price: 590.0
```

Explanation: The first version hard-coded two prices, computed tax and total inline, and printed the result twice. Refactoring extracted a calculate_total_price helper with proper documentation, then iterated over a list of prices. This groups related logic, avoids copy-paste, and makes the calculation reusable for any price. Future changes (different tax rate, currency formatting) now live in one place.

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

• **Task**: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.

Prompt: Refactor the code by Creating constants at the top of the file. Replace all hardcoded occurrences in calculations with these constants. Ensure the code remains functional and is easier to maintain.

Code:

```python
# # Legacy script with hardcoded values
# print("Area of Circle:", 3.14159 * (7 ** 2))
# print("Circumference of Circle:", 2 * 3.14159 * 7)
# Refactor the code by Creating constants at the top of the file. Replace all hardcoded occurrences in
# calculations with these constants.Ensure the code remains functional and is easier to maintain.
PI = 3.14159  # Define a constant for the value of pi
RADIUS = 7  # Define a constant for the radius of the circle
def calculate_circle_properties(radius):
    """
    Calculate the area and circumference of a circle given its radius.

    Parameters:
    radius (float): The radius of the circle.

    Returns:
    tuple: A tuple containing the area and circumference of the circle.
    """
    area = PI * (radius ** 2)  # Calculate area using the formula A = πr^2
    circumference = 2 * PI * radius  # Calculate circumference using the formula C = 2πr
    return area, circumference  # Return the calculated area and circumference as a tuple
# Calculate the properties of the circle using the defined radius
area, circumference = calculate_circle_properties(RADIUS)  # Call the function to calculate area and circumference
print(f"Area of Circle: {area}")  # Print the calculated area of the circle
print(f"Circumference of Circle: {circumference}")  # Print the calculated circumference of the circle
```

Output:

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation: Original lines crammed π (3.14159) and a radius of 7 directly into area/circumference formulas. The refactor defines PI and RADIUS constants at the top and uses a calculate_circle_properties function with a docstring. By naming the magic numbers you make the code self-documenting and maintainable, and the function encapsulates the geometry. Changing the radius or π value becomes trivial.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

• **Task**: Use AI to improve readability by renaming unclear variables and adding inline comments.

Prompt: Refactor and Replace vague names with meaningful ones. Add comments where logic is not obvious. Keep functionality exactly the same.

Code:

```python
# Legacy script with poor variable names
# a = 10
# b = 20
# c = a * b / 2
# print(c)
# Refactor and Replace vague names with meaningful ones.Add comments where logic is not obvious.Keep functionality exactly the same.
def calculate_triangle_area(base, height):
    """
    Calculate the area of a triangle given its base and height.

    Parameters:
    base (float): The base length of the triangle.
    height (float): The height of the triangle.

    Returns:
    float: The area of the triangle.
    """
    area = (base * height) / 2  # Calculate area using the formula A = (base * height) / 2
    return area  # Return the calculated area
# Define the base and height of the triangle
base_length = 10  # Base length of the triangle
height_length = 20  # Height of the triangle
# Calculate the area of the triangle using the defined base and height
triangle_area = calculate_triangle_area(base_length, height_length)  # Call the function to calculate the area of the triangle
print(f"Area of Triangle: {triangle_area}")  # Print the calculated area of the triangle
```

Output:

```
Area of Triangle: 100.0
```

Explanation: Legacy code used a, b, and c with no comments to compute and print a value; you had to guess it was a triangle's area. The refactored version uses base_length and height_length, wraps the formula in calculate_triangle_area, and comments it. These meaningful names and the docstring remove ambiguity and make the math immediately understandable. The behaviour is identical but the intent is obvious.

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

• **Task**: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

Prompt: Refactor and identify redundant conditional checks.Replace them with a reusable function.Ensure grading logic remains unchanged.

Code:

```python
# code:
# marks = 85
# if marks >= 90: print("Grade A")
# elif marks >= 75: print("Grade B")
# else:
# print("Grade C")
# marks = 72
# if marks >= 90:
# print("Grade A")
# elif marks >= 75:
# print("Grade B")
# else:
# print("Grade C")
# Refactor and identify redundant conditional checks.Replace them with a reusable function.Ensure grading logic remains unchanged.
def get_grade(marks):
    """
    Determine the grade based on the marks obtained.
    Parameters:
    marks (float): The marks obtained by the student.
    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:  # Check if marks are greater than or equal to 90
        return "Grade A"  # Return Grade A for marks 90 and above
    elif marks >= 75:  # Check if marks are greater than or equal to 75
        return "Grade B"  # Return Grade B for marks between 75 and 89
    else:
        return "Grade C"  # Return Grade C for marks below 75
# List of marks to evaluate
marks_list = [85, 72]
# Loop through each mark and determine the grade
for marks in marks_list:
    grade = get_grade(marks)  # Call the function to get the grade for the given marks
    print(f"Marks: {marks}, {grade}")  # Print the marks and the corresponding grade
```

Output:

```
Marks: 85, Grade B
Marks: 72, Grade C
```

Explanation: Two separate blocks each checked marks with the same condition chain (>=90, >=75, else) – duplicated and error-prone. Refactoring pulls the decision into a get_grade function with clear comments; the loop then just calls it for each mark. You've eliminated redundancy and encapsulated the grading rules so they're defined once.

The code now reads like "for each mark, get the grade" which mirrors the human thought process.

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

• **Task**: Use AI to refactor procedural input–processing logic into functions.

Prompt: Refactor and Identify input, processing, and output sections.Convert each into a separate function.Improve code readability without changing behavior.

Code:

```
# Sample Legacy Code:
# num = int(input("Enter number: "))
# square = num * num
# print("Square:", square)
# Refactor and Identify input, processing, and output sections.Convert each into a separate function.
# Improve code readability without changing behavior.
def get_user_input():
    """
    Prompt the user to enter a number and return it as an integer.
    Returns:
    int: The number entered by the user.
    """
    return int(input("Enter number: "))  # Get user input and convert it to an integer
def calculate_square(number):
    """Calculate the square of a given number.
    Parameters:
    number (int): The number to be squared.

    Returns:
    int: The square of the given number.
    """
    return number * number  # Return the square of the given number
def display_output(square):
    """
    Display the calculated square.
    Parameters:
    square (int): The square to be displayed.
    """
    print("Square:", square)  # Print the calculated square
# Main execution flow
user_number = get_user_input()  # Get user input
square_result = calculate_square(user_number)  # Calculate the square of the user input
display_output(square_result)  # Display the calculated square
```

Output:

```
Enter number: 23
Square: 529
```

Explanation: The sample code mingled input, processing and output all in one place: read a number, square it, print it. Refactoring splits those concerns into three functions— get_user_input, calculate_square and

display_output—each documented. This modular structure makes the flow easier to follow and the pieces easier to test or reuse. It's a classic "separate I-O from logic" cleanup.

Task Description #8 (Refactoring – Optimizing List Processing)

• **Task**: Use AI to refactor inefficient list processing logic.

Prompt: Refactor and replace manual loops with list comprehensions or built-in functions.Ensure output remains identical.

Code:

```
# Sample Legacy Code:
# nums = [1, 2, 3, 4, 5]
# squares = []
# for n in nums:
# squares.append(n * n)
# print(squares)
# Refactor and replace manual loops with list comprehensions or built-in functions.Ensure output remains identical.
def calculate_squares(numbers):
    """
    Calculate the squares of a list of numbers.

    Parameters:
    numbers (list): A list of numbers to be squared.

    Returns:
    list: A list containing the squares of the input numbers.
    """
    return [n * n for n in numbers]  # Use a list comprehension to calculate squares
# List of numbers to be squared
nums = [1, 2, 3, 4, 5]
# Calculate the squares of the numbers using the defined function
squares = calculate_squares(nums)  # Call the function to calculate squares
print(f"squares: {squares}")  # Print the list of squares
```

Output:

```
squares: [1, 4, 9, 16, 25]
```

Explanation: Originally a loop appended each number's square to a list in three lines. The refactor replaces that manual loop with a single list comprehension inside calculate_squares, complete with a docstring. The result is shorter, more idiomatic Python, and plainly expresses "take each number and square it". Output stays the same, but the implementation is cleaner and leverages built-ins.